Entrega 2: Núcleo do domínio

Projeto 2: Urna eletrônica. Texto para colar na seção da Entrega 2 do Relatório do Projeto.

2.1 UML atualizado

Colar aqui a imagem do diagrama (arquivo uml_entrega2.png). O que mudou em relação à Entrega 1 e por quê:

Classe	O que mudou	Por quê
Pessoa	Ganhou o método abstrato `getPapel()`.	Para a classe abstrata ter um método abstrato e permitir polimorfismo entre Eleitor e Candidato.
Eleitor	`registrarVoto()` virou privado. Entraram `votar(Urna, Candidato)`, `votar(Urna, int)`, `votarBranco(Urna)`, `jaVotou()` e `verificarSePodeVotar()`.	O eleitor confere o `jaVotou` antes de pedir o voto à urna e só o marca depois que a urna aceitou. Assim o voto e o `jaVotou` andam juntos (RN2.1 e RN2.5) e o `jaVotou` só muda por método do próprio eleitor.
Candidato	Sobrescreve `getPapel()`. Mantém numero, cargo, partido e foto.	Polimorfismo: cada classe filha diz o seu papel.
Urna	`votar(int)` devolve o `TipoVoto`. `contabilizarVotos()` devolve um `Map<Candidato, Integer>`. Entraram `contarBrancos()`, `contarNulos()`, `getTotalVotos()` e `getVotos()` (lista não modificável).	A tela precisa saber quando o voto virou nulo para avisar o eleitor. A apuração precisa de brancos, nulos e total. A lista de votos não pode ser alterada de fora.
Voto	O construtor deixou de ser público (visível só no pacote `modelo`). Entraram `getTipo()` e `getCandidato()`.	Só a Urna cria votos (RN2.4).
Exceções	As quatro exceções viraram classes que estendem `Exception`, no pacote `modelo`.	O compilador obriga a tratar cada uma.

Não mudaram: as heranças (Pessoa, Eleitor e Candidato), a agregação da urna com os candidatos, a composição da urna com os votos, o enum `TipoVoto` e a ausência de ligação entre Voto e Eleitor (RN2.2).

2.2 Conceitos de OO aplicados

Conceito	Onde aparece (classe / método)	Explicação
Classe abstrata e método abstrato	`Pessoa` / `getPapel()`	`Pessoa` é abstrata e declara `getPapel()` sem corpo. Não dá para criar uma Pessoa direto: ela sempre é eleitor ou candidato.
Herança	`Eleitor extends Pessoa` e `Candidato extends Eleitor` (construtores com `super(...)`)	Herança em três níveis: o candidato reaproveita tudo do eleitor, que reaproveita nome e CPF da pessoa.
Polimorfismo	`Eleitor.getPapel()` e `Candidato.getPapel()` (`@Override`); a classe `Demo` percorre uma `List<Eleitor>`	O mesmo `getPapel()` responde "Eleitor" ou "Candidato" conforme o objeto, sem usar `if` para descobrir o tipo.
Sobrecarga de método ou construtor	`Urna.votar(Candidato)` e `Urna.votar(int)`; `Eleitor.votar(Urna, Candidato)` e `Eleitor.votar(Urna, int)`	Mesmo nome para votar direto no candidato ou pelo número digitado.
Encapsulamento	`Eleitor.jaVotou` e `registrarVoto()`; `Urna.getVotos()`; construtor de `Voto`	O `jaVotou` é privado e só muda por método do eleitor. A lista de votos sai como lista não modificável. Só a urna cria votos.
Princípio de Liskov	`Main` e `Demo`: `eleitores.addAll(candidatos)`	Um `Candidato` entra onde se espera um `Eleitor` e `votar(...)` funciona igual, sem quebrar nada.

Conceito	Onde aparece (classe / método)	Explicação
Agregação ou composição	`Urna` (construtor e lista `votos`)	Agregação: a lista de candidatos chega pelo construtor e os candidatos existem sem a urna. Composição: a lista de votos nasce dentro da urna e só ela cria os votos.
Enum	`TipoVoto`	Conjunto fixo de valores (VALIDO, BRANCO e NULO), evita erro de digitação.

2.3 Exceções customizadas

Exceção	Quando é lançada	Onde é tratada
`EleitorJaVotouException`	`Eleitor.verificarSePodeVotar()`, quando o `jaVotou` já é verdadeiro (em `votar` e `votarBranco`)	`Main.votar()` e `Demo`, no bloco `catch`: mostram a mensagem e o programa continua
`UrnaEncerradaException`	`Urna.verificarAberta()`, ao votar numa urna encerrada	`Main.votar()` e `Demo`
`EleitorNaoEncontradoException`	`Main.buscarEleitor()`, quando o título não está cadastrado	`Main.votar()` e `Demo`
`CandidatoInvalidoException`	`Urna.votar(Candidato)`, quando o candidato não pertence à urna	`Demo`

Todas estendem `Exception` (verificadas): o compilador obriga a tratar ou declarar com `throws`.

2.4 Evidências de execução

Saída do `app.Demo`, uma demonstração automática que não pede nada no teclado. Ela mostra cadastro em listas, voto válido, branco e nulo, as quatro exceções tratadas, o encerramento e a apuração. Como o voto é secreto, a saída mostra só quem votou, nunca em quem: votos em candidato, em branco e nulos aparecem apenas nos totais da apuração.

```
=== DEMONSTRAÇÃO - Urna da seção 001 ===

Cadastro (polimorfismo: getPapel() muda conforme o objeto):
  [Candidato] Ana Souza (10) - Prefeito - Partido X
  [Candidato] Bruno Lima (20) - Prefeito - Partido Y
  [Candidato] Carla Dias (30) - Prefeito - Partido Z
  [Eleitor] Diego Alves (título 2001)
  [Eleitor] Elisa Rocha (título 2002)
  [Eleitor] Fábio Costa (título 2003)

--- Votação (o voto é secreto: só aparece quem votou, nunca em quem) ---
OK: Diego Alves votou.
OK: Elisa Rocha votou.
OK: Fábio Costa votou.
OK: Ana Souza votou.

--- Exceções tratadas ---
ERRO (EleitorJaVotouException): Diego Alves já votou.
ERRO (EleitorNaoEncontradoException): Título 9999 não cadastrado.
ERRO (CandidatoInvalidoException): Esse candidato não pertence a esta urna.
OK: Carla Dias votou.

--- Apuração (urna encerrada) ---
Ana Souza (10): 1
Bruno Lima (20): 2
Carla Dias (30): 0
Brancos: 1
Nulos: 1
Total de votos: 5
Conferência: 3 + 1 + 1 = 5

--- Tentativa depois de encerrar ---
ERRO (UrnaEncerradaException): A urna da seção 001 está encerrada.
```

A conferência fecha: votos nos candidatos (3) + brancos (1) + nulos (1) = 5 votos, o mesmo número de eleitores que votaram.

Como rodar: compilar com `javac -d out src/modelo/*.java src/app/*.java` e executar `java -cp out app.Demo`. O `app.Main` é a versão interativa, com menu no console (votar, encerrar e apurar).

Requisitos cobertos nesta entrega, em listas: RF2.1 a RF2.5 e RN2.1 a RN2.4.